

Universidade Federal de Uberlândia
Faculdade de Computação

Engenharia de Software
Prof. Fabiano Azevedo Dorça

Arquitetura de Software
Parte 03

Arquitetura de Software

- **Comunicação em arquiteturas distribuídas de software (ex. microsserviços):**
 - comunicação **síncrona** via HTTP (REST)
 - comunicação **assíncrona**
 - **via gRPC** (Google Remote Procedure Call) ou
 - **Message Queues** (RabbitMQ, Apache Kafka, Apache ActiveMQ)
 - **Pub/Sub** (Google Cloud Pub/Sub, Apache Kafka, Amazon Simple Notification Service (SNS), Azure Event Grid entre outros.)

Arquitetura de Software

- REST (Representational State Transfer - transferência de estado representacional)
 - é um **estilo de arquitetura client-server para sistemas distribuídos**,
 - define um conjunto de princípios para a comunicação entre sistemas, especialmente na web.
 - É um modelo que visa a simplicidade, escalabilidade e flexibilidade, utilizando os protocolos HTTP existentes para troca de informações.
 - Ideal para sistemas web **client-server** em também para sistemas baseados em arquitetura de **microserviços**.

Arquitetura de Software

- **Stateless**

- Cada **solicitação do cliente para o servidor deve conter todas as informações necessárias para o servidor** processá-la, sem depender de informações de solicitações anteriores.
- Nesta arquitetura, a interação entre cliente e servidor é padronizada através de uma **interface única, usando HTTP e formatos como JSON ou XML.**

- **O REST é amplamente suportado por diversas bibliotecas e ferramentas, facilitando o desenvolvimento de aplicações.**

Arquitetura de Software

- Em resumo:
 - REST é um estilo arquitetural que **define princípios** para a criação de sistemas distribuídos, na web,
 - focando em simplicidade, escalabilidade e flexibilidade.
 - a comunicação via REST é tipicamente **síncrona**.
 - Isso significa que o **cliente envia uma solicitação e aguarda uma resposta** do servidor antes de continuar com outras tarefas
 - Essa resposta pode ser o resultado da operação solicitada ou um código de erro, indicando que algo deu errado.



A high performance, open source universal RPC framework

- gRPC (<https://grpc.io/>)
 - O **gRPC é um framework moderno, de código aberto e de alto desempenho**, para Chamadas de Procedimento Remoto (RPC) que pode ser executado em qualquer ambiente.
 - inicialmente desenvolvido pelo Google e agora é gerenciado pela Cloud Native Computing Foundation (CNCF).

Arquitetura de Software

- gRPC é uma estrutura que facilita a comunicação entre diferentes serviços e aplicações, especialmente em arquiteturas de **microserviços**.
 - Utiliza **HTTP/2** para transporte e Protocol Buffers (um formato de serialização de dados eficiente) para a comunicação, resultando em **alta performance e baixa latência**.
 - **Ideal para comunicação entre microserviços**, permitindo a criação de sistemas distribuídos de forma mais fácil e eficiente.

Arquitetura de Software

- é uma estrutura de código aberto para **chamadas de procedimento remoto (RPC)** de alto desempenho,
 - permite que um cliente chame métodos em um servidor remoto como se fossem métodos locais, **facilitando a criação de sistemas distribuídos**.
 - é **otimizado** para alta taxa de transferência e baixa latência, ideal para aplicações que exigem comunicação rápida e eficiente.
 - suporta streaming de dados, permitindo a comunicação bidirecional entre cliente e servidor, o que é ideal para aplicações em tempo real.

Arquitetura de Software

- suporta diversas linguagens de programação, incluindo Java, Python, Go, C#, etc.
- O protocolo **HTTP/2 não é síncrono.**
 - **Ele é assíncrono e multiplexado**, o que significa que várias solicitações e respostas podem ocorrer simultaneamente na mesma conexão TCP.
 - **Ao contrário do HTTP/1.1, que usava conexões separadas para cada solicitação,**
 - o HTTP/2 permite que múltiplas solicitações e respostas sejam enviadas e recebidas na mesma conexão, melhorando o desempenho e reduzindo a latência.

Arquitetura de Software

- Exemplos de uso:
 - **Comunicação entre microsserviços** em arquiteturas nativas da nuvem.
 - Comunicação entre clientes móveis e servidores de back-end.
 - Aplicações que exigem baixa latência e alta taxa de transferência.
- gRPC
 - é uma estrutura poderosa e eficiente para a **construção de sistemas distribuídos,**
 - **especialmente em arquiteturas de microsserviços**, oferecendo benefícios como alto desempenho, comunicação eficiente e suporte multiplataforma.

Arquitetura de Software

- Vantagens do gRPC:
 - **Desempenho superior:**
 - O gRPC é mais rápido e eficiente em comparação com outras tecnologias RPC, como REST.
 - **Recursos avançados:**
 - O gRPC oferece recursos como streaming, autenticação e segurança.
 - **Aplicações em tempo real:**
 - O suporte a streaming do gRPC o torna ideal para aplicações que exigem comunicação em tempo real, como jogos online e sistemas de chat.
 - **Flexibilidade:**
 - O gRPC suporta diferentes linguagens e plataformas, permitindo a criação de sistemas heterogêneos.

Arquitetura de Software

- **Aplicações que exigem alta performance**: Em cenários onde a latência e a taxa de transferência são críticas, o gRPC oferece um desempenho superior.
- **Comunicação entre microsserviços**: O gRPC é uma excelente opção para comunicação de alto desempenho entre microsserviços em arquiteturas modernas
- Em resumo, **o gRPC é uma poderosa ferramenta para construir sistemas distribuídos** de alto desempenho, oferecendo vantagens significativas sobre outras tecnologias RPC, especialmente em ambientes de microsserviços e aplicações em tempo real.

Arquitetura de Software

Used by



gRPC is a CNCF incubation project



Arquitetura de Software

- Arquiteturas **orientadas a mensagens**
- A comunicação ente **clientes e servidores** é mediada por um terceiro serviço,
 - Uma **fila de mensagens** em que
 - os **clientes atuam como produtores** de informações, inserindo mensagens na fila (FIFO).
 - Os **servidores atuam como consumidores** de mensagens, retirando mensagens da fila e processando-as.
 - Uma **mensagem é um registro, ou um objeto**, com um conjunto de dados.

Arquitetura de Software

- Isto permite **comunicação assíncrona**;
 - Ao colocar a informação na fila, o cliente está liberado para continuar seu processamento.
 - Fila de mensagens são muito usadas na construção de **sistemas distribuídos**;
 - **Filas de mensagens** são também conhecidas como **brokers de mensagens**
 - Permitem **comunicação assíncrona** entre clientes e servidores

Arquitetura de Software

- Viabiliza **duas formas de desacoplamento** entre os componentes de uma aplicação distribuída:
 - **Desacoplamento no espaço**: clientes não precisam conhecer os servidores, e vice-versa.
 - O cliente é exclusivamente um produtor de informação, e não precisa saber quem vai consumir essa informação.
 - O mesmo vale para servidores.

Arquitetura de Software

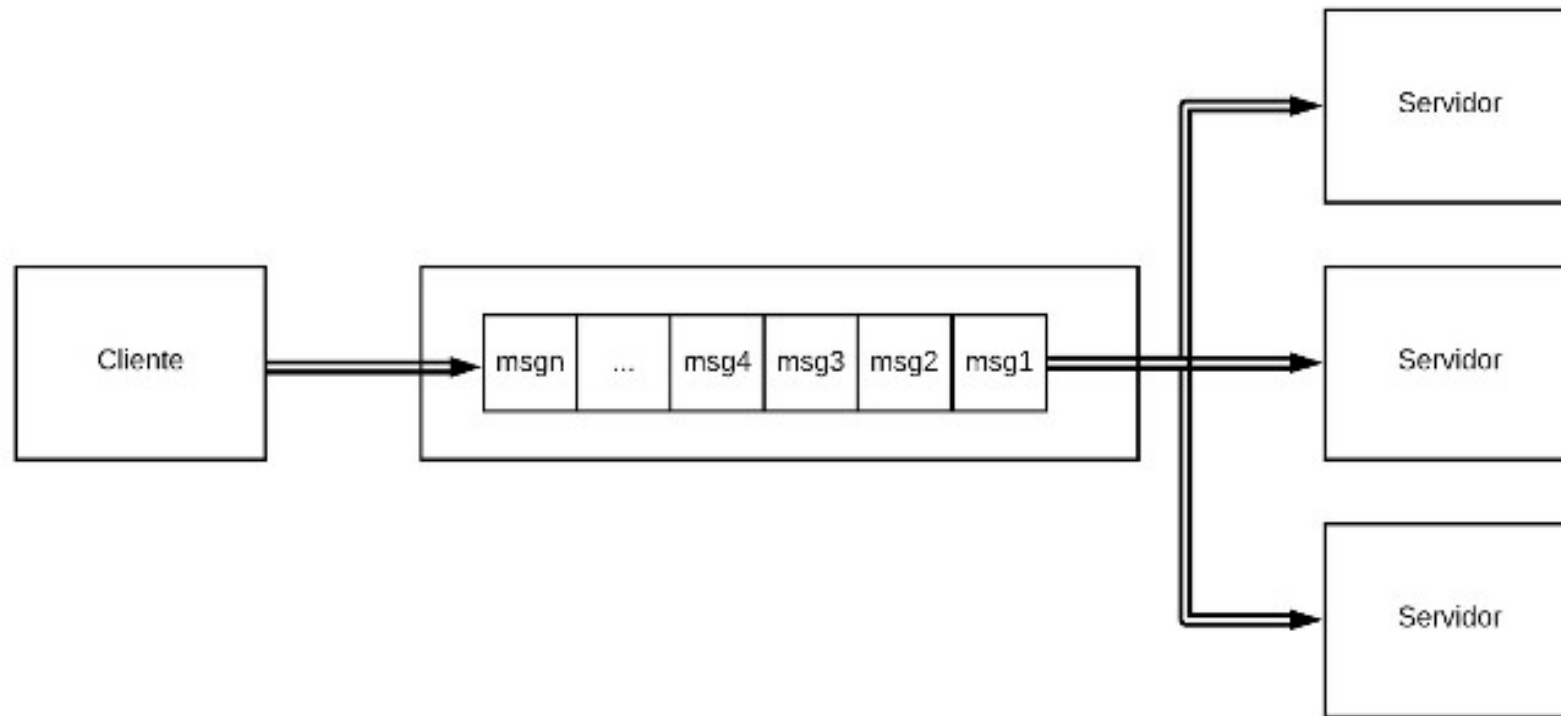
- **Desacoplamento no tempo**: clientes e servidores não precisam estar disponíveis para se comunicarem.
 - Se o servidor estiver fora do ar, os clientes podem continuar produzindo mensagens e colocando-as na fila.
 - Quando o servidor voltar a funcionar, ele irá processar as mensagens.

Isto torna a solução robusta a falhas. Queda do servidor não tem impacto nos clientes.

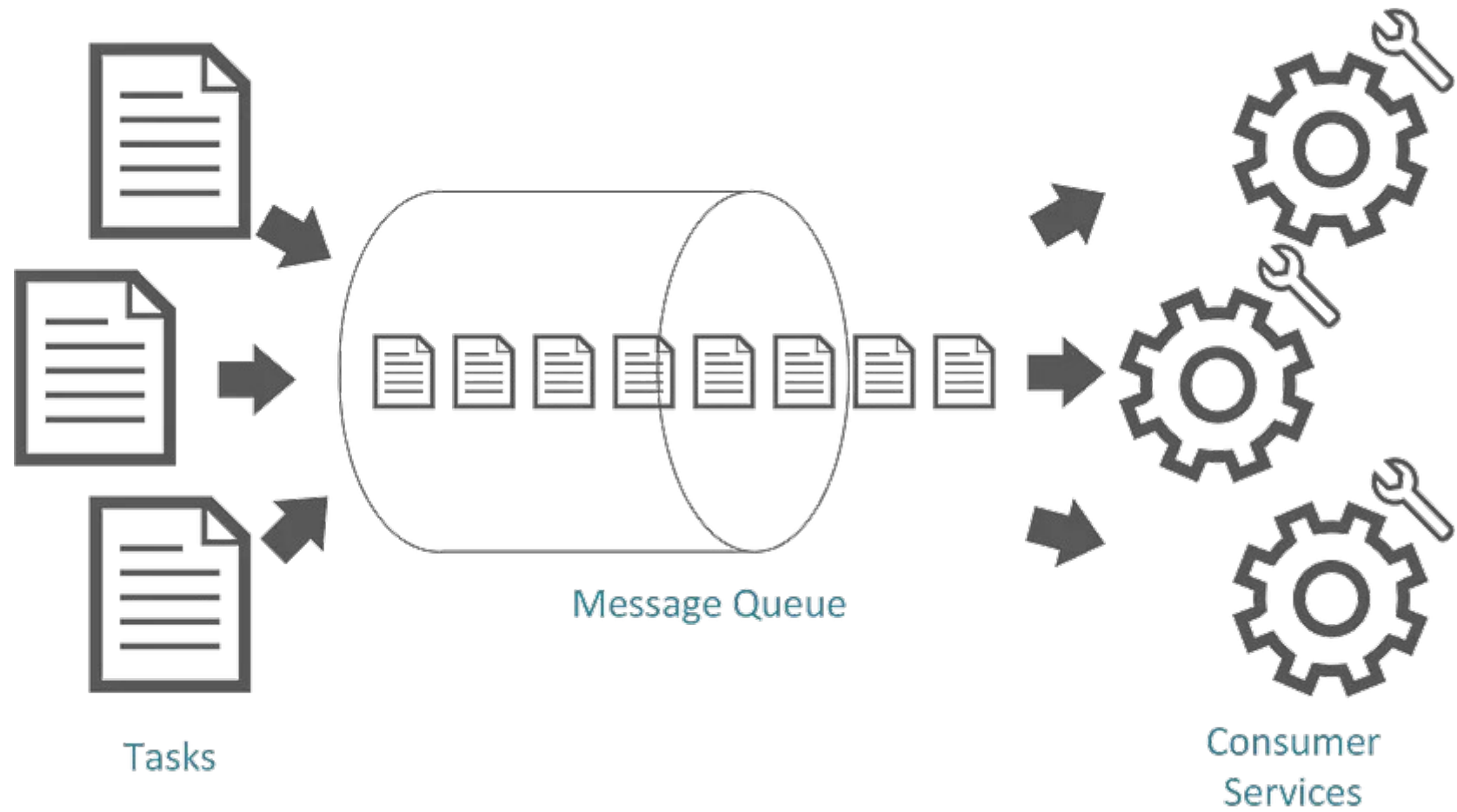
- Permite escalar mais facilmente, configurando múltiplos servidores consumidores de mensagens.

Arquitetura de Software

- Fila de mensagens com vários consumidores



Arquitetura de Software



Arquitetura de Software

- **RabbitMQ** (MQ – Message Queue)
 - é um sistema de **fila de mensagens** de código aberto altamente popular que implementa o **Advanced Message Queuing Protocol (AMQP)**.
 - se trata de um **intermediário** de comunicação
 - **comunicação assíncrona** é amplamente usada quando não há necessidade de esperar uma resposta.

Arquitetura de Software

- Ele permite que os aplicativos se comuniquem entre si por meio de um sistema de mensagens altamente flexível, robusto e escalável.
- **RabbitMQ** também oferece **suporte a vários protocolos** de mensagens, como AMQP, MQTT e STOMP.
- **RabbitMQ** oferece suporte a uma ampla variedade de linguagens e protocolos legados.

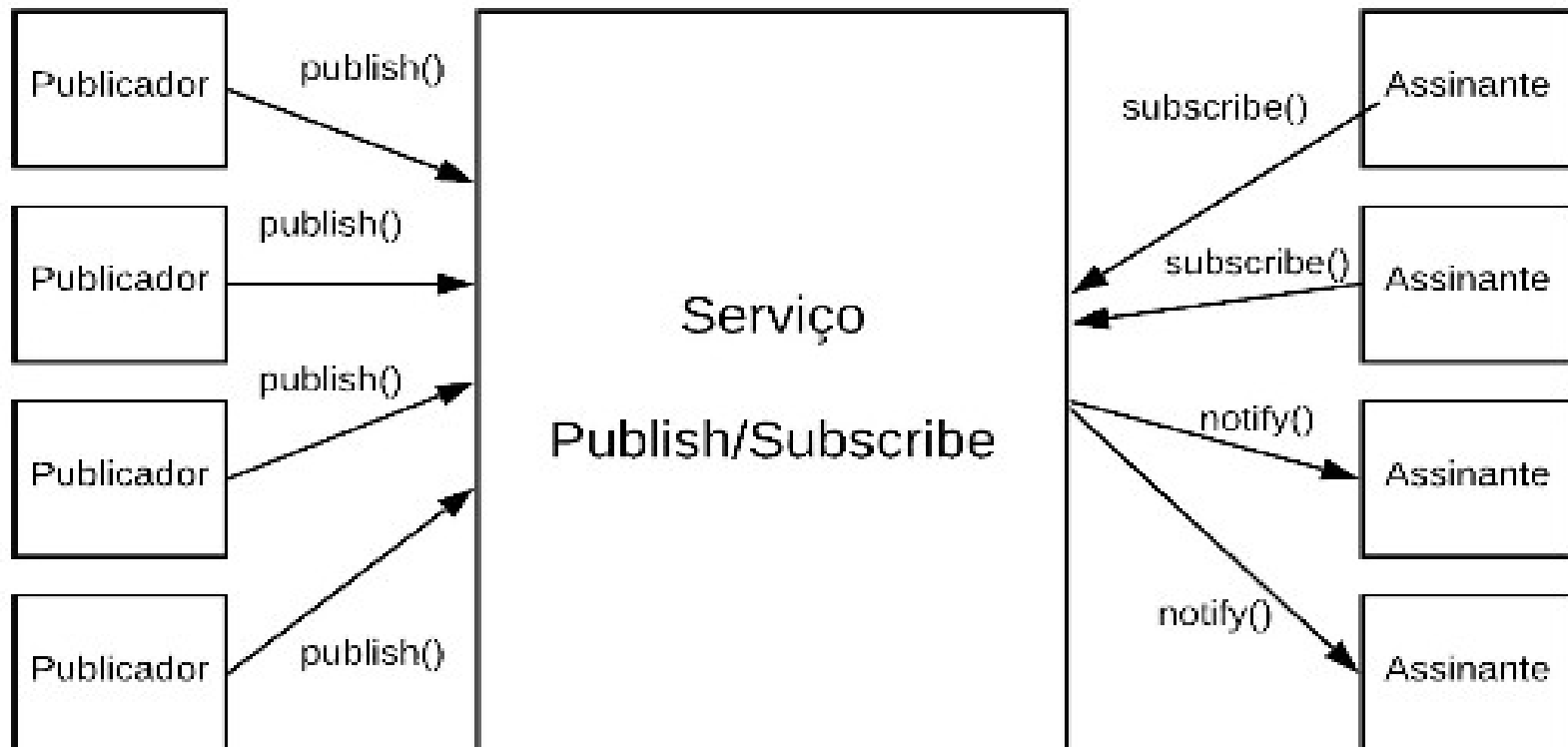
Arquitetura de Software

- **Arquitetura publish/subscribe (Padrão Pub/sub)**
 - Arquitetura orientada a **eventos**
 - Pub/Sub, ou Publicação/Subscrição, é um **padrão de comunicação assíncrona** em sistemas distribuídos que desacopla os componentes que produzem mensagens (publicadores) daqueles que as consomem (assinantes).
 - Os publicadores enviam mensagens para tópicos, e os assinantes se inscrevem nesses tópicos para receber as mensagens relevantes

Arquitetura de Software

- Publicação e Assinatura

- Quando um **evento** é publicado, todos os seus assinantes são notificados.



Arquitetura de Software

- Os **remetentes (publishers)** enviam eventos, ou também conhecido como “mensagens”, para notificar alguma criação, atualização, deleção, seja o que for, sobre seu domínio,
- Estas notificações são encaminhadas para os **destinatários (subscribers)**, que optam por receber estes eventos e os utilizam para realizar operações dentro de seu domínio também.

Arquitetura de Software

- O interessante desse modelo, é que a comunicação **não é feita diretamente entre o Publisher e o Subscriber,**
 - caso mais entidades optam por receber estes eventos, ficaria inviável ao longo do tempo do publicador realizar esse gerenciamento de saber para quem deve enviar estes eventos, toda vez que algum domínio opta por receber ou deixa de querer receber.

Arquitetura de Software

- É também conhecida como arquitetura orientada a evento, ou **broker de eventos**.
 - Funciona como um **barramento** por onde devem trafegar todos os eventos.
 - É uma arquitetura **para sistemas distribuídos**.
 - Produtores e assinantes são processos distintos, e na maioria das vezes, distribuídos

Arquitetura de Software

- Benefícios:
 - **Desacoplamento**: Permite que componentes se comuniquem sem conhecimento direto uns dos outros, facilitando a escalabilidade e flexibilidade do sistema.
 - **Escalabilidade**: Pub/Sub é projetado para lidar com um grande volume de mensagens e um grande número de assinantes, suportando sistemas distribuídos complexos.
 - **Confiabilidade**: O serviço Pub/Sub garante que as mensagens sejam entregues aos assinantes, mesmo que haja falhas ou interrupções.
 - **Assincronicidade**: Permite que publicadores e assinantes operem independentemente, melhorando a eficiência e a capacidade de resposta do sistema.

- Exemplo de Uso:
 - Integração de Sistemas
 - Pub/Sub pode ser usado para **integrar diferentes sistemas e serviços**, permitindo que eles compartilhem informações de forma assíncrona.
- Exemplos de serviços de mensageria que implementam o modelo Pub/Sub
 - incluem o **Google Cloud Pub/Sub, Apache Kafka, Amazon Simple Notification Service (SNS), Azure Event Grid entre outros.**

Arquitetura de Software

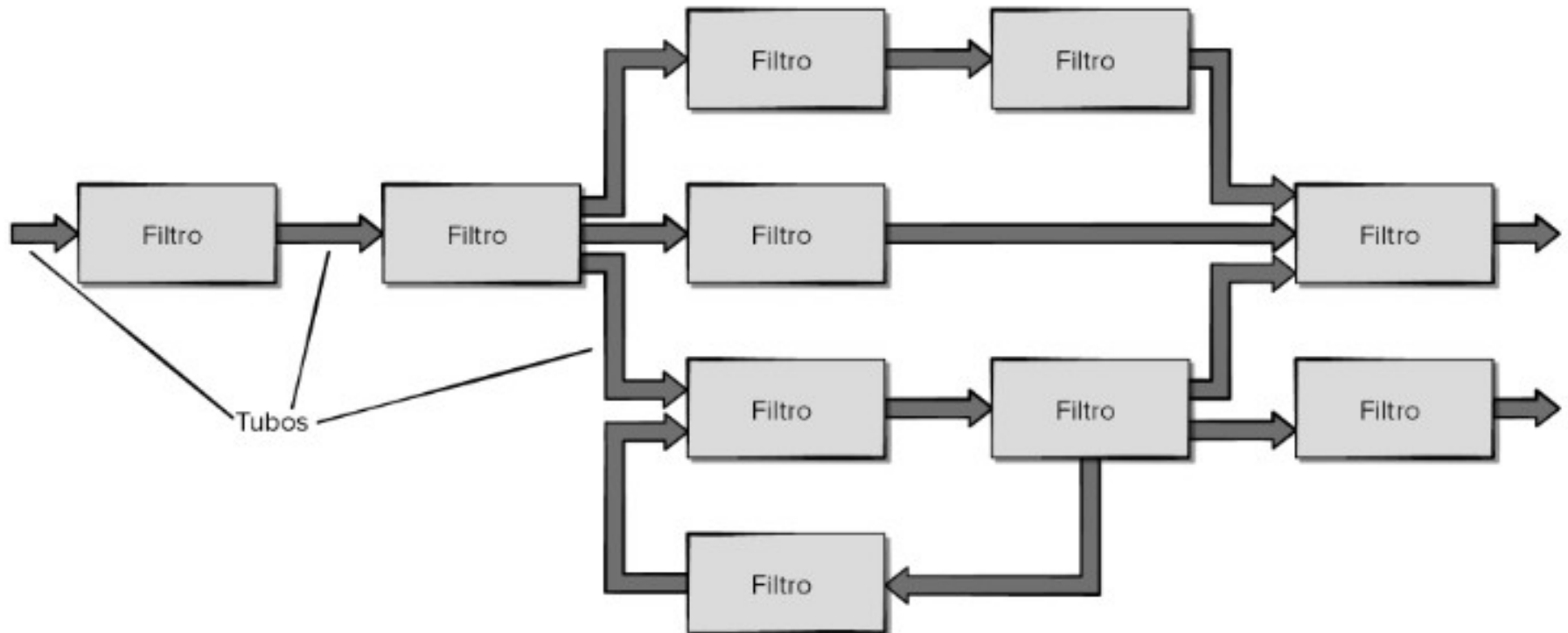
- Pipes and filters (tubos e filtros)
 - Um fluxo de dados processado por uma série de componentes (filtros) conectados por canais (pipes), permitindo a composição de funcionalidades complexas.
 - Arquitetura orientada a dados

Arquitetura de Software

- Programas (filtros) devem processar dados recebidos na entrada e gerar uma saída
- Os filtros são conectados por meios de pipes, que agem como buffers, armazenando a saída de um filtro enquanto não é lida pelo próximo filtro
- Os filtros não precisam conhecer seus antecessores e sucessores – flexibilidade
- Permite variadas combinações de filtros independentes (programas).

Arquitetura de Software

- Arquitetura de fluxo de dados.



Arquitetura de Software

- Essa arquitetura se aplica quando
 - ***dados de entrada devem ser transformados por meio de uma série de componentes*** computacionais ou de manipulação em dados de saída.
- Tem um conjunto de componentes, denominado filtros, conectados por tubos que transmitem dados de um componente para o seguinte.

Arquitetura de Software

- **Cada filtro trabalha de modo independente** dos componentes que se encontram acima e abaixo deles,
 - é projetado para esperar a entrada de dados de determinada forma e produz saída de dados (para o filtro seguinte) da forma especificada.
- Entretanto, o filtro não precisa conhecer o funcionamento interno de seus filtros vizinhos.
- Um sistema de processamento de dados pode utilizar o estilo pipes e filtros para processar grandes volumes de informação.

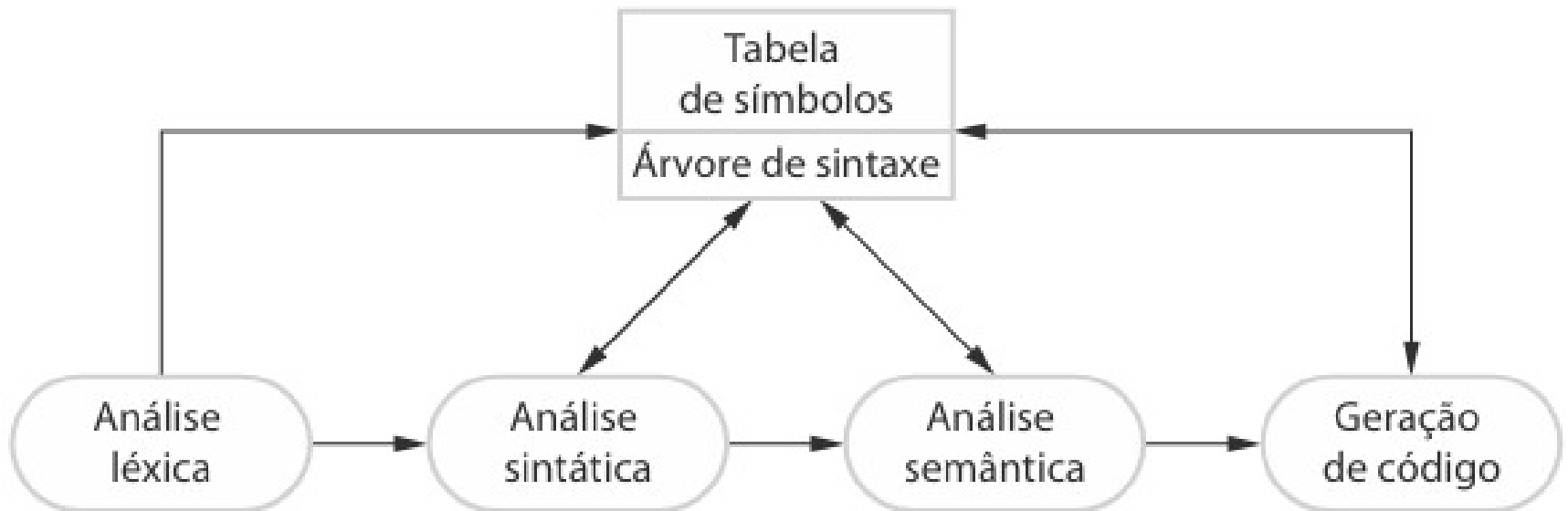
Arquitetura de Software

- Exemplo:
 - Comandos de sistemas Unix,
 - `ls | grep "csv" | sort`
 - No caso do Unix, as entradas e saídas são sempre arquivos texto.



Arquitetura de Software

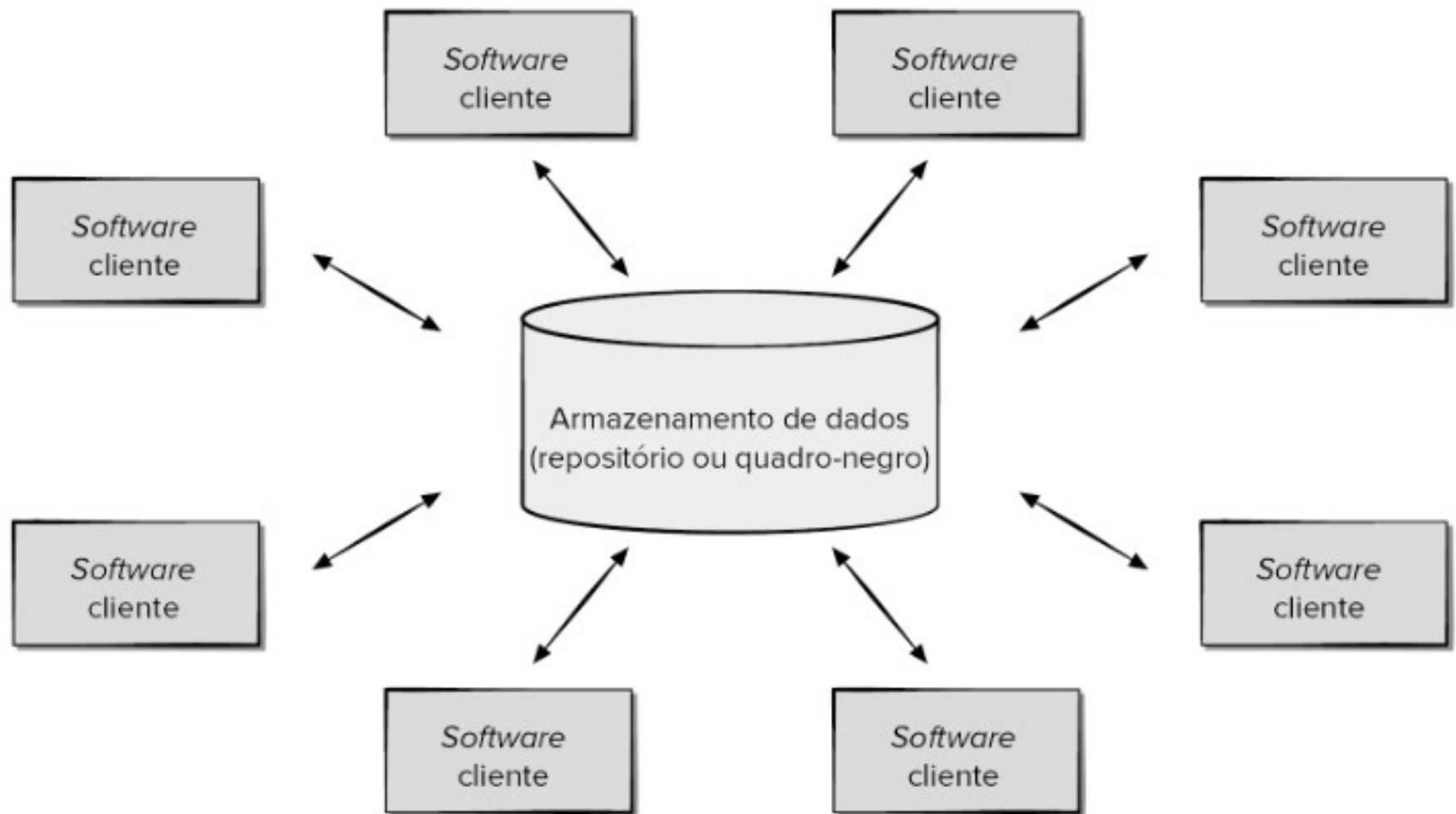
- Arquitetura do compilador em duto e filtro



Arquitetura de Software

- Arquitetura centralizada em dados.
 - Um **repositório de dados (p. ex., um arquivo ou banco de dados) reside no centro dessa arquitetura** e é acessado frequentemente por outros componentes que atualizam, acrescentam, excluem ou modificam de alguma outra maneira os dados contidos no repositório.

Arquitetura de Software



Arquitetura de Software

- O software cliente acessa um repositório central. **O repositório de dados é passivo.**
- Ou seja, o software cliente acessa os dados independentemente de quaisquer alterações nos dados ou das ações de outros softwares clientes.
- Uma variação dessa abordagem transforma o repositório em um “quadro-negro” que envia notificações ao software cliente quando os dados de seu interesse mudam.

Arquitetura de Software

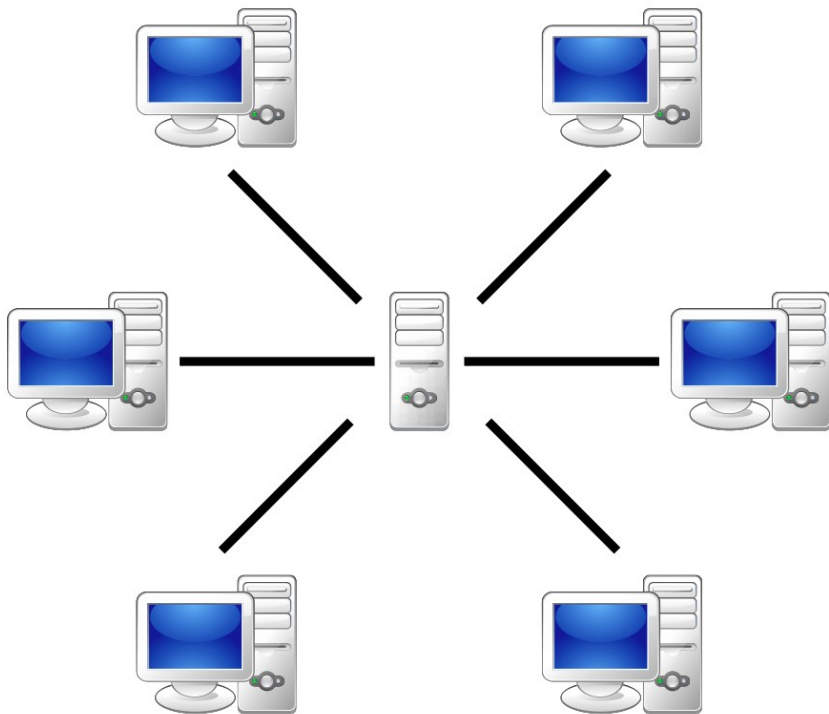
- Este padrão arquitetura é também chamado **repositório**

Nome	Repositório
Descrição	Todos os dados em um sistema são gerenciados em um repositório central, acessível a todos os componentes do sistema. Os componentes não interagem diretamente, apenas por meio do repositório.
Quando é usado	Você deve usar esse padrão quando tem um sistema no qual grandes volumes de informações são gerados e precisam ser armazenados por um longo tempo. Você também pode usá-lo em sistemas dirigidos a dados, nos quais a inclusão dos dados no repositório dispara uma ação ou ferramenta.
Vantagens	Os componentes podem ser independentes — eles não precisam saber da existência de outros componentes. As alterações feitas a um componente podem propagar-se para todos os outros. Todos os dados podem ser gerenciados de forma consistente (por exemplo, <i>backups</i> feitos ao mesmo tempo), pois tudo está em um só lugar.
Desvantagens	O repositório é um ponto único de falha, assim, problemas no repositório podem afetar todo o sistema. Pode haver ineficiências na organização de toda a comunicação através do repositório. Distribuir o repositório através de vários computadores pode ser difícil.

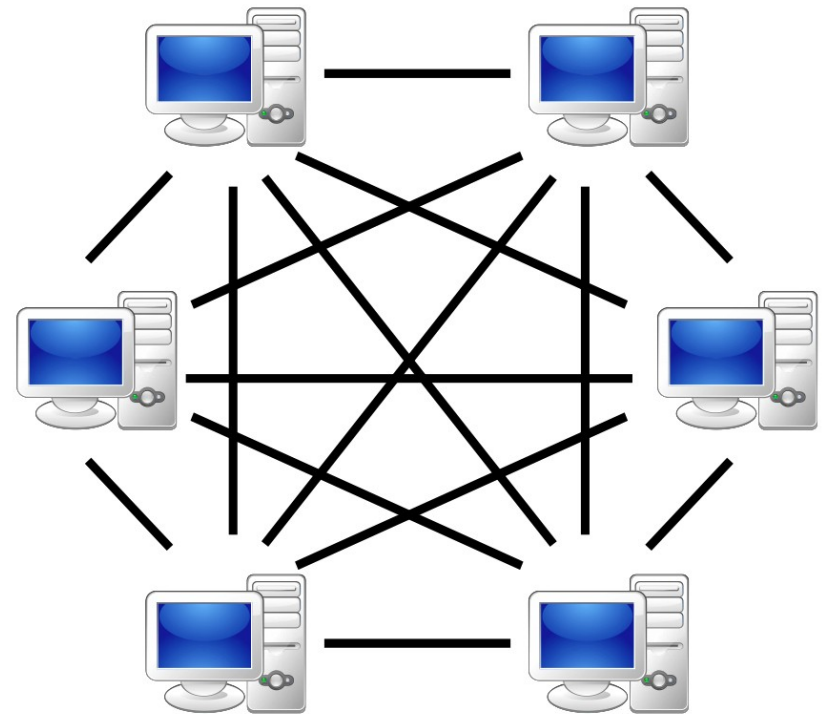
Arquitetura de Software

- Peer-to-peer (P2P), ou par-a-par,
 - é um modelo arquitetural onde cada nó (computador) atua tanto como cliente quanto como servidor, compartilhando recursos e informações diretamente entre si sem depender de um servidor central.

Arquitetura de Software



Server-based



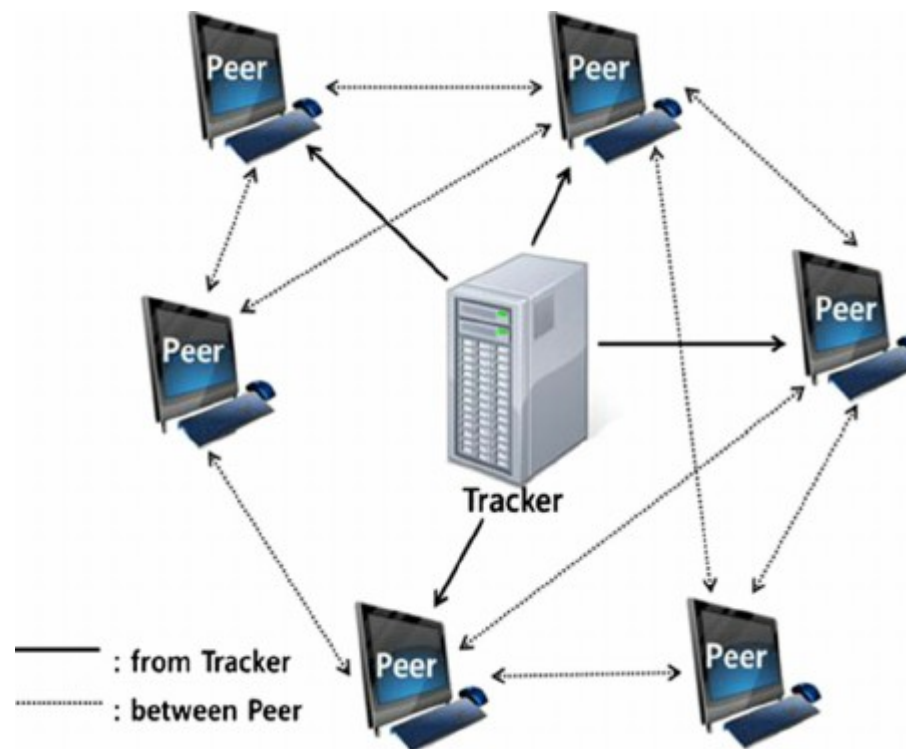
P2P-network

Arquitetura de Software

- Características:
 - **Descentralização**: Não há um servidor central gerenciando a rede. Cada nó é igual e se conecta diretamente aos outros.
 - **Compartilhamento de Recursos**: Nodos compartilham recursos como arquivos, poder de processamento e armazenamento entre si.
 - **Escalabilidade**: Redes P2P podem ser facilmente expandidas adicionando mais nós.
 - **Resiliência**: Se um nó falha, outros nós ainda podem continuar a funcionar.
 - **Segurança**: Gerenciar a segurança em redes P2P pode ser mais desafiador, pois não há um ponto central para aplicar políticas.

Arquitetura de Software

- **Bittorrent**
- O protocolo criado por Bram Cohen permitia que os pares se comunicassem diretamente através de uma porta TCP, mas contavam com rastreadores centrais para registrar a localização / disponibilidade de arquivos e coordenar os usuários.
- **Jogos online:** Alguns jogos online utilizam conexões diretas entre jogadores, baseadas em P2P.

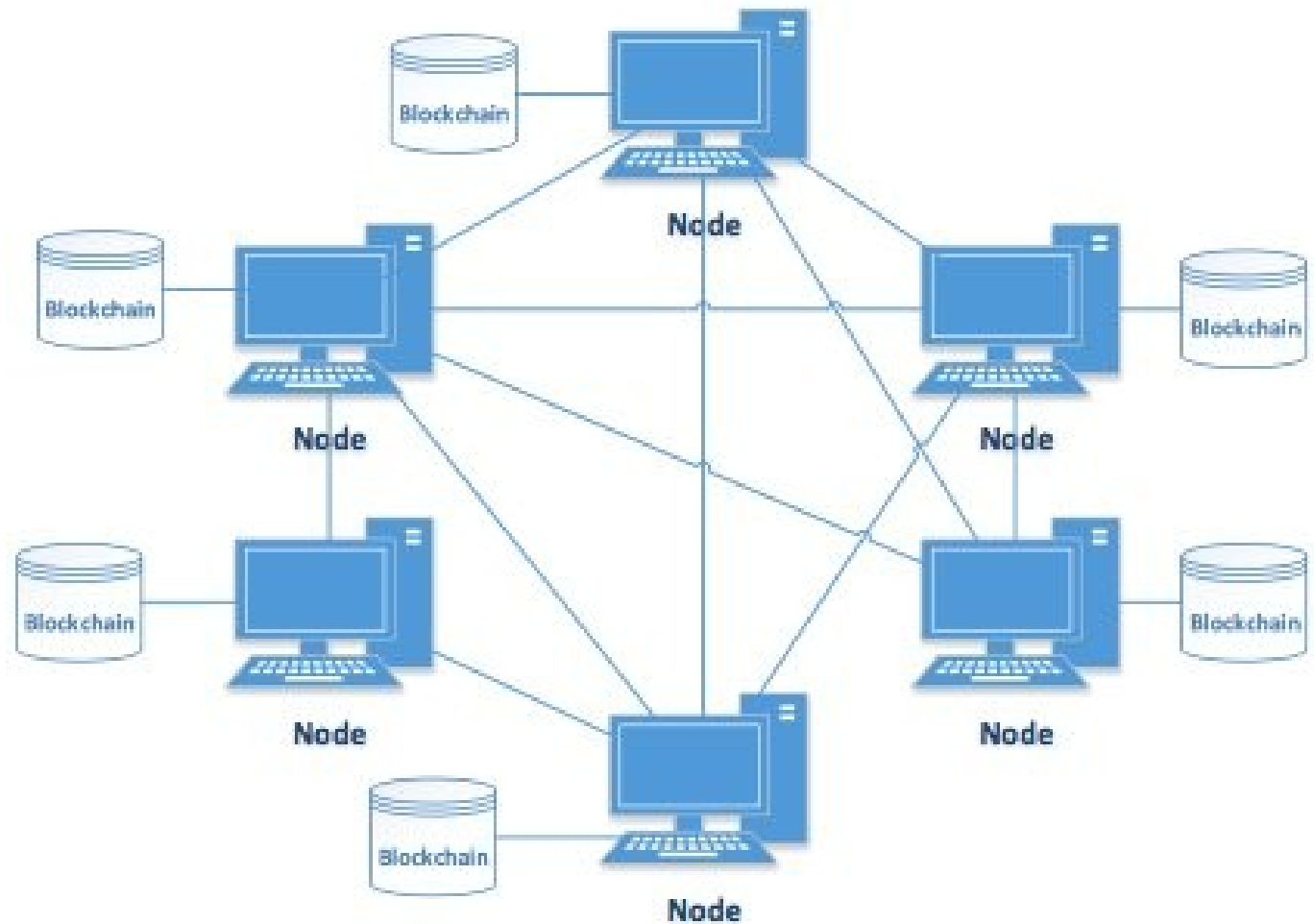


Arquitetura de Software

- Em **redes blockchain**, P2P (Peer-to-Peer)
 - arquitetura onde todos os participantes (nós) têm o mesmo nível de acesso e interagem diretamente uns com os outros, sem um servidor central.
- Essa descentralização é fundamental para a segurança e a confiabilidade das blockchains,
 - pois cada nó mantém uma cópia do livro-razão e valida as transações.
 - blockchain nada mais é do que um livro de registros codificado em cadeia e que não permite alterações.

Arquitetura de Software

- **Blockchain P2P é a base para criptomoedas**
- Ex. Bitcoin

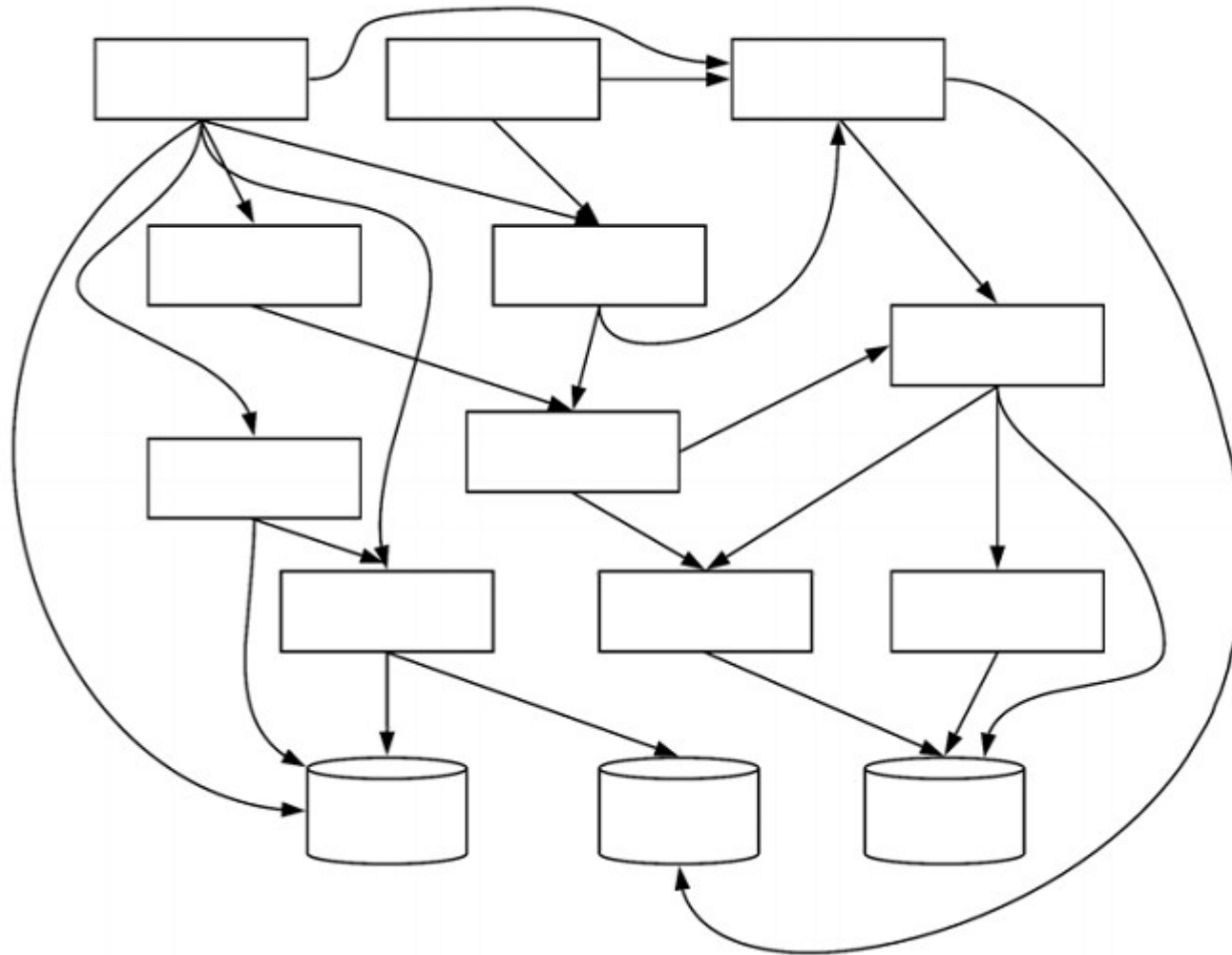


Arquitetura de Software

- Anti-padrões arquiteturais
- Big ball of mud (grande bola de lama)
 - Qualquer módulo comunica-se com qualquer outro módulo.
 - Não possui uma arquitetura definida.
 - Explosão no número de dependências
 - A manutenção do sistema torna-se difícil e arriscada.

Arquitetura de Software

- Big ball of mud (grande bola de lama)



Arquitetura de Software

- Importância dos estilos arquiteturais:
 - **Compreensão**: Estilos arquiteturais facilitam a compreensão da estrutura e organização de sistemas complexos.
 - **Reutilização**: Permitem a reutilização de componentes e padrões em diferentes projetos.
 - **Qualidade**: Influenciam atributos de qualidade como desempenho, segurança e manutenibilidade.
 - **Comunicação**: Facilitam a comunicação entre os membros da equipe de desenvolvimento.
 - **Decisão**: Orientam a escolha de tecnologias e ferramentas adequadas para o desenvolvimento.

Arquitetura de Software

- Dúvidas?

Exercícios

- Sobre RabbitMQ, analise as afirmativas abaixo e dê valores Verdadeiro (V) ou Falso (F).
- () RabbitMQ é um middleware de mensageria open-source que implementa o protocolo Advanced Message Queuing Protocol (AMQP).
- () O RabbitMQ é exclusivo para a linguagem de programação Java, não oferecendo suporte a outras linguagens de programação.
- () No RabbitMQ, os produtores são responsáveis por enviar mensagens para uma fila, enquanto os consumidores as recebem e processam.

- Assinale a alternativa que apresenta a sequência correta de cima para baixo.
- (A) F - F - F
- (B) F - V - F
- (C) V - F - V
- (D) V - V - V

- RESPOSTA
- LETRA C

- Nos serviços de mensageria, a comunicação síncrona via HTTP é mais adequada para cenários de alta concorrência do que a comunicação assíncrona.
- (C) Certo
- (E) Errado

- RESPOSTA
- (E) Errado

- Assinale a opção que apresenta exemplos de serviços de mensageria em que há agentes que publicam, conhecidos como publishers, e agentes que leem, conhecidos como consumers.
- (A) RabbitMQ e Kafka.
- (B) XML-HTTP Request.
- (C) ZooKeeper e WSDL.
- (D) SOAP e Ionic.
- (E) Data lake e Spark.

- RESPOSTA
- LETRA A